

Edgar Ali Guerra Villarreal - 22400600

Programación Web

Índice...

[JavaScript](#)

[Características Básicas](#)

[Estructuras de Control](#)

[Condicionales](#)

[Bucles](#)

[Tipos de Datos](#)

[Tipos Primitivos](#)

[Tipos de Referencia](#)

[Funciones](#)

[Declaración de Función](#)

[Expresión de Función](#)

[Funciones Flecha \(Arrow Functions\)](#)

[Objetos](#)

[Arrays](#)

[Uso con const](#)

[Eventos y Manipulación del DOM](#)

[Manipulación del DOM](#)

[Asincronía](#)

[Callbacks](#)

[Promesas](#)

[Async/Await](#)

JavaScript

JavaScript es un lenguaje de programación interpretado y versátil, pilar de la web. Se utiliza principalmente para crear páginas interactivas, desde validaciones de formularios y animaciones hasta aplicaciones web complejas, y también en el servidor mediante entornos como Node.js.

Características Básicas

JavaScript es un lenguaje de programación con las siguientes características fundamentales:

- **Interpretado:** El código se ejecuta línea por línea en el navegador o en el entorno de ejecución (como Node.js), sin necesidad de compilación previa.
- **Dinámico:** Las variables no tienen un tipo fijo; pueden contener diferentes tipos de datos en distintos momentos. Esto también permite que los arrays contengan valores de tipos mixtos .
- **Multiparadigma:** Soporta programación orientada a objetos, funcional e imperativa, lo que ofrece gran flexibilidad al desarrollador.
- **Lenguaje del lado del cliente:** Originalmente diseñado para ejecutarse en navegadores, permitiendo interactuar con el usuario y modificar el contenido de la página web.

Estructuras de Control

Condicionales

Las estructuras condicionales permiten ejecutar diferentes bloques de código según se cumpla o no una condición.

- **if...else:** Evalúa una condición y ejecuta un bloque si es verdadera, y otro si es falsa
- **switch:** Evalúa una expresión y ejecuta el bloque de código correspondiente al valor que coincida.

Bucles

Los bucles repiten un bloque de código mientras se cumpla una condición .

- **for:** Se utiliza cuando se conoce el número de iteraciones .
- **while:** Ejecuta el código mientras la condición sea verdadera .
- **do...while:** Similar a while, pero garantiza que el código se ejecute al menos una vez .
- **for...of:** Itera sobre valores de objetos iterables (como arrays) .
- **for...in:** Itera sobre las propiedades enumerables de un objeto .

Tipos de Datos

JavaScript divide los tipos de datos en dos categorías principales: primitivos y de referencia .

Tipos Primitivos

Los tipos primitivos se copian por valor. Cuando se asigna una variable primitiva a otra, se crea una copia independiente .

- **String:** Cadenas de texto
- **Number:** Números (enteros y decimales)
- **Boolean:** true o false
- **undefined:** Variable declarada pero sin valor
- **null:** Ausencia intencional de valor
- **Symbol (ES6):** Valor único e inmutable
- **BigInt:** Números enteros muy grandes

Tipos de Referencia

Los tipos de referencia se copian por referencia. La variable almacena una dirección en memoria que apunta al objeto, no el objeto en sí . Cuando se asigna un objeto a otra variable, ambas apuntan al mismo objeto en memoria .

- **Object:** Colección de propiedades con nombre
- **Array:** Lista ordenada de valores
- **Function:** Bloques de código reutilizables
- **Date:** Fechas y horas
- **RegExp:** Expresiones regulares

Funciones

Las funciones son bloques de código reutilizables. En JavaScript, hay tres formas principales de definir las.

Declaración de Función

Se definen con la palabra clave `function` y tienen hoisting, lo que permite llamarlas antes de su declaración.

```
function saludar(nombre) {  
  return `Hola, ${nombre}`;  
}  
console.log(saludar("Ana")); // "Hola, Ana"
```

Expresión de Función

Se asignan a una variable. No tienen hoisting, por lo que solo pueden llamarse después de ser definidas.

```
const saludar = function(nombre) {  
  return `Hola, ${nombre}`;  
};  
console.log(saludar("Luis")); // "Hola, Luis"
```

Funciones Flecha (Arrow Functions)

Introducidas en ES6, ofrecen una sintaxis más concisa y heredan el contexto `this` del ámbito padre.

```
// Con un parámetro (paréntesis opcionales)  
const sumaDos = num => num + 2;  
  
// Con múltiples parámetros  
const suma = (a, b) => a + b;  
  
// Con cuerpo en bloque (requiere return)  
const sumaDos = a => {  
  const total = a + 2;  
  return total;  
};
```

Objetos

Los objetos son colecciones de propiedades, donde cada propiedad tiene un nombre (clave) y un valor.

```
const persona = {
  nombre: "Sam",
  edad: 55,
  acceso: true
};

// Acceso con punto
console.log(persona.nombre); // "Sam"

// Acceso con corchetes (como array asociativo)
console.log(persona["edad"]); // 55

// Agregar nuevas propiedades
persona.hijos = ["Max", "Rosa"];
```

Arrays

Los arrays son objetos especializados para almacenar listas ordenadas de valores.

- Son dinámicos: pueden crecer o reducirse
- Pueden contener tipos mixtos (números, strings, booleanos, otros arrays, objetos)
- Se crean con corchetes [] o usando el constructor Array()

```
const numeros = [43, "hello", -2.5, true];
const matriz = [6, [5, 3, -2], "JavaScript"];

// Métodos comunes
numeros.push(100); // Agrega al final
numeros.pop();     // Elimina el último
console.log(numeros.length); // Tamaño del array
```

Uso con const

Al declarar un array con const, la variable siempre apunta al mismo array, pero su contenido puede modificarse.

```
const nombres = ["Sam", "Scott"];
nombres.push("Jack"); // Válido
nombres[0] = "Bob";   // Válido
// nombres = ["Nuevo"]; // Error: reasignación no permitida
```

Eventos y Manipulación del DOM

El DOM (Document Object Model) es la representación en forma de árbol de la estructura de una página web, y JavaScript puede manipularlo para cambiar el contenido, estilo y estructura de la página de forma dinámica.

Manipulación del DOM

```
// Seleccionar elementos
const elemento = document.getElementById("miId");
const elementos = document.querySelectorAll(".miClase");

// Modificar contenido
elemento.textContent = "Nuevo texto";
elemento.innerHTML = "<strong>Texto con HTML</strong>";

// Cambiar estilos
elemento.style.color = "blue";

// Crear y agregar elementos
const nuevo = document.createElement("div");
nuevo.textContent = "Elemento creado";
document.body.appendChild(nuevo);
```

Los eventos son acciones que ocurren en la página (clic, tecleo, movimiento del mouse). JavaScript permite "escuchar" estos eventos y ejecutar código en respuesta.

```
const boton = document.querySelector("#miBoton");
boton.addEventListener("click", function() {
    alert("¡Botón clickeado!");
});

// Con función flecha
boton.addEventListener("click", () => {
    console.log("Evento disparado");
});
```

Asincronía

JavaScript es single-threaded (un solo hilo de ejecución), pero maneja operaciones asíncronas mediante tres mecanismos principales.

Callbacks

Una función que se pasa como argumento y se ejecuta cuando una operación asíncrona completa.

```
function cargarDatos(callback) {
  setTimeout(() => {
    callback("Datos cargados");
  }, 1000);
}

cargarDatos((resultado) => {
  console.log(resultado);
});
```

Promesas

Introducidas en ES6, representan el resultado (éxito o fallo) de una operación asíncrona.

```
const promesa = new Promise((resolve, reject) => {
  // Operación asíncrona
  if (exito) {
    resolve("Resultado exitoso");
  } else {
    reject("Error en la operación");
  }
});

promesa
  .then(resultado => console.log(resultado))
  .catch(error => console.error(error));
```

Async/Await

Sintaxis introducida en ES8 que permite escribir código asíncrono con apariencia síncrona.

```
async function obtenerDatos() {
  try {
    const respuesta = await fetch("https://api.ejemplo.com/datos");
    const datos = await respuesta.json();
    console.log(datos);
  } catch (error) {
    console.error("Error:", error);
  }
}

// La función async siempre devuelve una Promesa
obtenerDatos();
```

